ROLE-BASED INVENTORY AND ASSET MANAGEMENT SYSTEM

A Mini Project Report

submitted by

APARNA P

(MES25MCA-2010)

to the APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the award of the Degree of

Master of Computer Applications



Department of Computer Applications
MES College of Engineering
Kuttippuram, Malappuram – 679582

October, 2026

Declaration

I undersigned hereby declare that the project report "ROLE-BASED INVENTORY AND ASSET MANAGEMENT SYSTEM" submitted for partial fulfilment of the requirements for the award of degree of Master of Computer Applications of the APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by me under supervision of Mr. Geevar C Zachariaz, Assistant Professor, Department of Computer Applications. This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

APARNA P
(MES25MCA-2010)
22/10/2026

DEPARTMENT OF COMPUTER APPLICATIONS

MES COLLEGE OF ENGINEERING, KUTTIPPURAM



CERTIFICATE

This is to certify that the report entitled ROLE-BASED INVENTORY AND ASSET MANAGEMENT SYSTEM is a bonafide record of the Mini Project work during the year 2026-27 carried out by APARNA P (MES25MCA-2010) submitted to the APJ Abdul Kalam Technological University, in partial fulfilment of the requirements for the award of the Master of Computer Applications, under my guidance and supervision. This report in any form has not been submitted to any other University or Institution for any purpose.

Internal Supervisor

Head of The Department

Acknowledgment

I express my sincere gratitude to our principal Dr. Rahumathuniza I for permitting and providing all facilities to carry out this project. I would like to express my sincere gratitude to my internal supervisor, Mr. Geevar C Zachariaz, for his invaluable guidance, support, and encouragement throughout the development of this mini project. I also extend my thanks to the Head of the Department Prof. Hyderali K and all faculty members of the Department of Computer Applications, MES College of Engineering, for providing the necessary resources and a conducive environment for learning and innovation.

I am deeply grateful to the faculty members and fellow students whose experiences with the manual asset management process inspired the vision behind the Role-Based Inventory and Asset Management System. Their insights and feedback shaped the direction of this project.

Finally I thank my family and friends for their unwavering support and motivation. Above all, I extend my heartfelt gratitude ot Almighty for giving me the strength and wisdom to complete this project.

Aparna P
(MES25MCA-2010)

Abstract

The Role-Based Inventory and Asset Management System is a web-based platform designed to simplify the management, monitoring and requisition of departmental equipment, hardware assets, and workplace tasks in educational institutions and enterprise environments. In most departments today, asset requests are submitted through emails or printed documents, allocations are handled manually, and equipment return and maintenance records are scattered across different places. This makes tracking an asset from purchase to disposal slow, inconsistent and difficult to audit.

The main objective of Role-Based Inventory and Asset Management System is to provide a single centralized platform with secure role-based access for the Administrator, Department Head, and Employee. Employees browse available stock, submit requisitions online, and return equipment when finished. The Administrator reviews requests, approves or rejects them, oversees low-stock threshold warnings, and dispatches workload-balanced tasks. Damaged equipment can be flagged through maintenance tickets, and inter-departmental collaboration issues are tracked to resolution. In addition, gamification rewards on-time task completion with achievement badges.

The system is developed using HTML, CSS and JavaScript with Tailwind CSS for the front end, and Python with the Django framework for the back end. SQLite is used as the development database, and Git and GitHub are used for version control. The project followed an Agile methodology with four sprints.

The system is expected to reduce paperwork and manual effort, improve communication between employees and administrators, prevent inventory loss, and make resource management more transparent and reliable.

Table of Contents

List of Figures viii
List of Tables ix
List of Appendices..... x
Chapter 1. Introduction 1
1.1 Motivation 1
1.2 Objectives..... 2
1.3 Contributions 2
1.4 Report Organization 3
Chapter 2. System Study 4
2.1 Existing System..... 4
2.2 Proposed System 5
2.3 Functionalities of the Proposed System 5
Chapter 3. Methodology..... 7
3.1 Introduction 7
3.2 Software Tools..... 7
3.3 Module Description..... 8
3.3.1 Administrator Module 8
3.3.2 Department Management Module 8
3.3.3 Asset Inventory Module 8
3.3.4 Employee Self-Service Module 8
3.3.5 Requisition and Returns Module 9
3.4 Workflow of the System 9
3.4.1 Registration and Login..... 9
3.4.2 Asset Catalog Setup 9
3.4.3 Requisition Submission 9
3.4.4 Administrative Review and Decision 9
3.4.5 Asset Return and Stock Restoration..... 10
3.4.6 Maintenance Defect Reporting 10
3.4.7 Task Workload Auto-Assignment 10
3.5 User Stories 10
3.6 Product Backlog 12
3.7 Sprint Backlog..... 13
3.8 Project Plan..... 14

3.9 Database Design 16
3.9.1 User Table 16
3.9.2 Asset Table..... 18
3.9.3 Asset Request Table 20
3.9.4 Department Table 21
3.9.5 Task and SubTask Table 22
3.9.6 Maintenance Issue Table 23
3.9.7 Department Issue Table 24
3.9.8 Award and Badge Table 25
Chapter 4. Results and Discussions..... 26
4.1 Results 26
4.2 Discussion 30
4.2.1 Analysis of Results..... 30
4.2.2 Role-Based Access Control..... 30
4.2.3 Requisition Workflow and Validation 30
4.2.4 Development Process..... 30
4.2.5 Limitations 31
Chapter 5. Conclusion and Future Scope..... 32
5.1 Conclusion..... 32
5.2 Future Scope..... 33
References..... 34
Appendix A: Data Flow Diagram 35
A.1 Level 0 DFD 35
A.2 Level 1 DFD 36
A.3 Level 2 DFD 37
A.4 Level 3 DFD 38
A.5 Level 4 DFD 39
Appendix B: Entity–Relationship Diagram 40
Appendix C: Source Code..... 41
C.1 Asset Data Model 41
C.2 Asset Requisition and Return Model 42
C.3 Task and SubTask Models 43
C.4 Admin Requisition Approval View 45
C.5 Return Asset Controller View 46
C.6 Workload Auto-Assignment View 47
Appendix D: Screenshots 49

D.1 Login Page..... 49
D.2 Admin Dashboard Page..... 49
D.3 Department Management Page..... 50
D.4 Asset List & Availability Page 50
D.5 Add New Asset Page 50
D.6 Employee Dashboard Page 51
D.7 Asset Requests & Approvals Page 51
D.8 Complaint & Issue Management Page 51
D.9 Task & Subtask Management Page 52
D.10 Performance Awards & Badges Page 52

List of Figures

Figure 4.1: Login Page..... 26
Figure 4.2: Admin Dashboard..... 27
Figure 4.3: Department Management..... 27
Figure 4.4: Asset List & Availability..... 28
Figure 4.5: Add New Asset Form..... 28
Figure 4.6: Employee Dashboard..... 29
Figure A.1: Level 0 DFD..... 35
Figure A.2: Level 1 DFD..... 36
Figure A.3: Level 2 DFD..... 37
Figure A.4: Level 3 DFD..... 38
Figure A.5: Level 4 DFD..... 39
Figure B.1: Entity–relationship diagram 40
Figure D.1: Login page of the system..... 49
Figure D.2: Admin dashboard with asset metrics.....49
Figure D.3: Department management directory..... 50
Figure D.4: Asset catalog with availability progress bars..... 50
Figure D.5: Add new asset form..... 50
Figure D.6: Employee workspace dashboard..... 51
Figure D.7: Asset requisition and return tracker..... 51
Figure D.8: Maintenance defect and issue tracker..... 51
Figure D.9: Task and milestone subtask board..... 52
Figure D.10: Gamified recognition badges and awards..... 52

List of Tables

Table 3.1: Software tools and technologies used for development..... 7
Table 3.2: User stories for review analysis and moderation..... 10
Table 3.3: Product backlog..... 12
Table 3.4: Sprint backlog and completion status..... 13
Table 3.5: Project schedule and task allocation..... 14
Table 3.6: User table schema and sample data..... 16
Table 3.7: Asset table schema and sample data..... 18
Table 3.8: Asset request table schema and sample data..... 20
Table 3.9: Department table schema and sample data..... 21
Table 3.10: Task and SubTask table schema and sample data.....22
Table 3.11: Maintenance issue table schema and sample data..... 23
Table 3.12: Department issue table schema and sample data..... 24
Table 3.13: Award and badge table schema and sample data..... 25

List of Appendices

Appendix A: Data Flow Diagram..... 35
Appendix B: Entity–Relationship Diagram..... 40
Appendix C: Source Code..... 41
Appendix D: Screenshots..... 49

Chapter 1. Introduction

Organizational assets and departmental inventory are an important part of institutional operations. Managing them involves several activities, including asset intake, departmental allocation, requisition submission, approval review, equipment return tracking, maintenance defect reporting, and record keeping. When these are handled through separate manual methods, tracking the complete inventory lifecycle becomes difficult.

This project proposes the Role-Based Inventory and Asset Management System, a web-based application that centralizes the management of organizational assets, requisitions, and departmental tasks. It provides separate functionalities for administrators and employees, based on their responsibilities. The system is implemented with HTML, CSS and JavaScript for the user interface and Python (Django) for server-side logic and database access. The interface is organised as role-based dashboards that are clean, responsive and easy to use.

1.1 Motivation

The main motivation for this project came from the difficulties observed in the way physical assets and inventory are handled in a typical department. The administrator must receive equipment requests, check stock availability, forward requests to department heads, allot items manually, monitor returns, and consolidate maintenance records. Employees, on the other hand, must approach several people through different channels to learn the availability of an asset or the status of their request. As a result, information is easily lost, delayed or duplicated. This observation motivated the development of a centralized platform that simplifies the requisition, monitoring and maintenance of organizational assets. The system improves communication between employees, administrators and department heads, and ensures transparent and efficient tracking throughout the asset lifecycle. By reducing paperwork and manual effort, it improves the overall efficiency and reliability of departmental asset management.

1.2 Objectives

The primary objective of Role-Based Inventory and Asset Management System is to develop a website with the following features:

- Provides secure role-based authentication for all users.
- Enables employees to browse available stock and submit asset requisitions.
- Allows the Administrator to manage departments, asset categories, and equipment.
- Enables the Administrator to review requisitions and grant approvals or rejections.
- Supports automated stock replenishment upon verified asset return.
- Allows employees to report physical defects and cross-departmental complaints.
- Supports workload-balanced task auto-assignment based on employee skills.
- Enables employees to check off milestone subtasks and track progress.
- Automatically rewards timely task completion with gamified achievement badges.
- Generates audit reports in PDF and Excel formats.
- Improves transparency, efficiency and coordination in asset management.

1.3 Contributions

Key contributions includes:

- A centralized web platform that replaces email- and paper-based inventory management.
- Secure role-based access for Administrator and Employee with automatic routing.
- An online requisition workflow with stock verification, approval, rejection, and return check-in.

- Automated inventory restoration that returns issued quantities to stock upon item check-in.
- A structured database storing asset details, requests, departments, tasks, subtasks, issues and awards.
- Export utilities for automated PDF and Excel inventory audit reports.

1.4 Report Organization

Chapter 2 reviews the existing system and the proposed solution. Chapter 3 describes the Agile methodology, development tools, modules, workflow, user stories, backlogs, project plan and database design. Chapter 4 presents the implemented interfaces and discusses the results and limitations. Chapter 5 gives the conclusion and future scope. The appendices contain the data flow diagrams, ER diagram, source code and screenshots.

Chapter 2. System Study

The system study examines how departmental inventory and assets are currently managed in the organization, and defines what the Role-Based Inventory and Asset Management System must provide. It looks at the present manual process, where the administrator receives requisitions, forwards them to department heads, collects their approvals, allocates equipment, logs maintenance notes and consolidates inventory counts, while employees must approach several people through different channels to learn the availability of an asset or check the status of their request. This often leads to lost, delayed or duplicated information. Based on this analysis, the study identifies the need for a centralized web-based platform that simplifies asset cataloging, requisition approval, return stock restoration and maintenance tracking. The system improves communication between employees, administrators and department heads, ensures transparent and efficient tracking, and reduces paperwork and manual effort.

2.1 Existing System

- Requisition submission: Requests are submitted through emails or printed paper slips.
- Asset allocation: The inventory coordinator allocates equipment manually in offline registers.
- Communication: Staff communicate through different platforms such as email and messaging applications.
- Return tracking: Checking in borrowed assets is difficult to monitor and maintain.
- Record keeping: Maintenance notes, defects and inventory reports are stored in different locations, and maintaining asset history is time-consuming.

- Stock visibility: There is no single platform where employees can check real-time asset availability or view low-stock warning alerts.
- Task management: Work is assigned verbally without workload balancing or milestone checklists.

2.2 Proposed System

The proposed system is a centralized web-based platform for complete asset and workflow management, with secure role-based access. Its main features are:

- Online asset catalog: Employees can view real-time availability and stock health bars.
- Requisition lifecycle: Employees submit requests with quantity and purpose; administrators approve or reject with notes.
- Automatic return replenishment: Returning an asset restores the available quantity immediately.
- Department management: Administrators create and oversee organizational departments.
- Maintenance reporting: Employees report damaged assets and inter-departmental issues.
- Workload auto-assignment: Tasks are automatically allocated to employees with matching skills and lowest workload.
- Gamification badges: On-time task completion earns recognition awards and badges.

2.3 Functionalities of the Proposed System

- Administrator: Manage assets, create departments, approve/reject requisitions, dispatch tasks, resolve maintenance issues, and grant awards.
- Employee: Browse inventory, request equipment, return checked-out items, report defects, complete subtasks, and earn badges.

- Department Head / Staff: Review issues received by their unit, update ticket status, and monitor departmental asset allocation.
- Authentication: Secure role-based login and registration for all system users.
- Inventory Management: Real-time stock counts, category classification, and automated low-stock warnings.
- Document & Audit Management: Export comprehensive inventory and asset audit reports in PDF and Excel formats.

Chapter 3. Methodology

3.1 Introduction

An Agile methodology was adopted for developing the Role-Based Inventory and Asset Management System. Agile promotes iterative development, continuous feedback and adaptive planning, which suits a user-centered resource management system. The project was divided into four sprints, each focusing on specific functionality such as authentication, department management, asset cataloging, requisition lifecycle, return restoration, maintenance ticketing, and gamified task assignment. This gave flexibility during development, allowed continuous improvement based on feedback from stakeholders, and ensured that requirements were implemented in an organized manner.

3.2 Software Tools

Table 3.1 lists the software tools and technologies used for development.

Table 3.1: Software tools and technologies used for development

Component	**Tool / Technology**
Operating System	Windows 11
Front End	JavaScript, HTML, CSS, Tailwind CSS
Back End	Python
Framework	Django
Database	SQLite
IDE	Visual Studio Code
Version Control	Git and GitHub

3.3 Module Description

The system is divided into modular components to ensure maintainability and scalability.

3.3.1 Administrator Module

This module allows the administrator to control the overall inventory and operational process. The administrator can log in, view real-time KPI metrics, receive low-stock alerts, manage asset records, create departments, and handle staff assignments.

3.3.2 Department Management Module

This module allows management of organizational units. Administrators can create, edit, and delete departments, allocate assets to specific units, and monitor employee headcounts and ticket loads.

3.3.3 Asset Inventory Module

This module manages the physical hardware and consumable equipment. It tracks asset names, categories, descriptions, total quantities, available quantities, and low-stock alert thresholds.

3.3.4 Employee Self-Service Module

This module allows employees to register, log in, browse real-time asset availability, submit checkout requests with justification notes, and view their assigned tasks and awards.

3.3.5 Requisition and Returns Module

This module governs the checkout pipeline. Administrators review pending requests to approve or reject them. When an approved asset is returned by an employee, the system automatically increments available stock.

3.4 Workflow of the System

The Role-Based Inventory and Asset Management System follows a sequence of well-defined stages. Each stage passes its output to the next, starting from user authentication and ending with task completion and award recognition.

3.4.1 Registration and Login

Employees register in the system and log in with their credentials. Administrators log in and are automatically routed to the executive dashboard, while employees are directed to their self-service workspace.

3.4.2 Asset Catalog Setup

The administrator configures inventory categories and inputs asset records with total counts and low-stock alert thresholds.

3.4.3 Requisition Submission

Employees browse the catalog, select available items, specify requested quantities and justification notes, and submit checkout requisitions.

3.4.4 Administrative Review and Decision

The administrator reviews pending requests, verifies stock sufficiency, and approves or rejects the requisition with administrative feedback notes.

3.4.5 Asset Return and Stock Restoration

When an employee completes their work with an approved item, clicking 'Return Asset' checks back the equipment and automatically replenishes available stock.

3.4.6 Maintenance Defect Reporting

Staff report hardware damage or inter-departmental requests. Department heads update tickets to In Progress or Resolved.

3.4.7 Task Workload Auto-Assignment

Tasks are created with subtask checklists and can be auto-assigned to the employee with the required skill and lowest pending workload hours.

3.5 User Stories

Table 3.2 summarizes the user and administrator requirements and their intended benefits.

Table 3.2: User stories for review analysis and moderation

Id	As a type of user	I want to	So that i can
1	User	Login to the system	Securely access my role dashboard
2	Employee	Register an account	Obtain authorized system credentials
3	Employee	Browse available inventory	Identify equipment available for request
4	Employee	Submit an asset request	Obtain needed hardware for project work
5	Employee	View my request status	Know whether requests are approved or rejected
6	Employee	Return approved assets	Check back in equipment and restore stock

7	Administrator	View admin dashboard	Monitor KPI metrics and stock health
8	Administrator	Receive low stock alerts	Initiate timely procurement orders
9	Administrator	Add new inventory assets	Make new equipment available in catalog
10	Administrator	Create categories via modal	Organize equipment without page reload
11	Administrator	Approve asset requests	Issue equipment to verified employees
12	Administrator	Reject asset requests	Provide feedback explaining denial reasons
13	Administrator	Create departments	Structure organizational business units
14	Administrator	Edit and delete departments	Maintain accurate department records
15	Employee	Report asset defect/damage	Notify technicians of broken hardware
16	Employee	Raise departmental ticket	Request assistance from other departments
17	Department	Update ticket status	Mark complaints as In Progress or Resolved
18	Administrator	Create tasks with subtasks	Delegate operational responsibilities
19	Administrator	Auto-assign tasks	Balance workload across skilled employees
20	Employee	Toggle milestone subtasks	Track progress toward task completion
21	Employee	Complete tasks on time	Earn automated timely completion badges
22	Administrator	Grant custom awards	Recognize outstanding employee performance
23	Employee	View my badge showcase	See personal achievements and medals

24	Administrator	Export PDF and Excel reports	Provide official audit documentation
25	Administrator	Manage staff departments	Reassign personnel across departments

3.6 Product Backlog

Table 3.3 presents the product backlog, including task priorities, estimates and completion status.

Table 3.3: Product backlog with priorities and estimated effort

ID	Name	Priority	Estimate(hrs)	Status
1	Requirement Analysis	High	2	Completed
2	System Design	High	2	Completed
3	Database/ER Diagram	High	3	Completed
4	UI/Template Design	Medium	2	Completed
5	Django Project Setup	High	2	Completed
6	Authentication and Roles	High	2	Completed
7	Initial Testing	Medium	2	Completed
8	Employee Registration View	High	3	Completed
9	Admin Dashboard KPI Cards	High	3	Completed
10	Department Management CRUD	High	2	Completed
11	Asset Inventory Catalog	High	2	Completed
12	Low Stock Alert Calculation	High	2	Completed
13	Inline Category Creation Modal	Medium	2	Completed
14	Asset Request Submission	High	2	Completed
15	Approval and Rejection Views	High	2	Completed
16	Return Asset & Stock Replenishment	Medium	2	Completed
17	Employee Dashboard View	Medium	2	Completed
18	Maintenance Defect Reporting	Medium	2	Completed

19	Cross-Department Ticketing	High	2	Completed
20	Ticket Status Update Form	High	3	Completed
21	Task Creation with Subtasks	Medium	2	Completed
22	Workload Auto-Assignment Engine	High	4	Completed
23	Subtask Interactive Toggling	Medium	2	Completed
24	Timely Completion Badge Engine	Medium	2	Completed
25	Custom Award Granting Interface	Medium	2	Completed
26	System Integration and Testing	High	3	Completed

3.7 Sprint Backlog

Table 3.4 organizes the development work into four sprints and records their completion status.

Table 3.4: Sprint backlog and completion status

Sprint	**Task**	**Status**
1	Requirement Analysis	Completed
1	System Design	Completed
1	Database / ER Design	Completed
1	UI / Template Design	Completed
1	Django Project Setup	Completed
1	Authentication and Roles	Completed
1	Initial Testing	Completed
2	Department Management CRUD	Completed
2	Asset Catalog and Availability	Completed
2	Low Stock Alert Calculation	Completed
2	Category Modal Creation	Completed
2	Employee Dashboard Layout	Completed

2	Requisition Submission Form	Completed
3	Administrative Approval View	Completed
3	Rejection Decision and Notes	Completed
3	Return Asset Stock Restoration	Completed
3	Maintenance Defect Tickets	Completed
3	Cross-Departmental Issue Flow	Completed
3	Ticket Status Updates	Completed
3	Task Creation with Subtasks	Completed
4	Workload Auto-Assignment Logic	Completed
4	Subtask Interactive Checkoff	Completed
4	Automated Achievement Badges	Completed
4	Custom Award Granting Interface	Completed
4	PDF and Excel Export Engines	Completed
4	System Integration and Testing	Completed

3.8 Project Plan

Table 3.5 presents the project schedule, user-story allocation and recorded task durations.

Table 3.5: Project schedule and task allocation

User Story ID	Task Name	Start Date	End Date	Duration (Days)	Status
1	Sprint 1	24/07/2026	24/07/2026	15	Completed
2		25/07/2026	27/07/2026		Completed
3		28/07/2026	28/07/2026		Completed
4		29/07/2026	31/07/2026		Completed

5		01/08/2026	03/08/2026		Completed
6		04/08/2026	04/08/2026		Completed
7		05/08/2026	07/08/2026		Completed
8	Sprint 2	08/08/2026	11/08/2026	15	Completed
9		12/08/2026	14/08/2026		Completed
10		15/08/2026	18/08/2026		Completed
11		19/08/2026	21/08/2026		Completed
12		22/08/2026	25/08/2026		Completed
13		26/08/2026	28/08/2026		Completed
14	Sprint 3	29/08/2026	31/08/2026		Completed
15		01/09/2026	01/09/2026		Completed
16		02/09/2026	04/09/2026		Completed
17		05/09/2026	08/09/2026		Completed
18		09/09/2026	11/09/2026		Completed
19		12/09/2026	15/09/2026		Completed
20		16/09/2026	18/09/2026		Completed
21	Sprint 4	19/09/2026	21/09/2026		Completed
22		22/09/2026	22/09/2026		Completed
23		23/09/2026	25/09/2026		Completed
24		26/09/2026	28/09/2026		Completed
25		29/09/2026	30/09/2026		Completed
26		01/10/2026	02/10/2026		Completed

3.9 Database Design

The database design utilizes SQLite integrated through Django ORM to store and manage user credentials, department structures, equipment inventory, requisition logs, and maintenance records. The detailed specifications for each table in the schema are outlined below.

3.9.1 User Table

Table 3.6 stores employee and administrator details, login information, and user roles.

Table 3.6: User table schema and sample data

Field Name	Data Type	Key/Constraint	Description	Sample Data
id	INTEGER	PK, Auto Increment	Unique user ID	18
username	VARCHAR(150)	UNIQUE, NOT NULL	Login username	admin
password	VARCHAR(128)	NOT NULL	Hashed password	pbkdf2_sha256$87...
first_name	VARCHAR(150)	NOT NULL	First name	Aparna
last_name	VARCHAR(150)	NOT NULL	Last name	P
email	VARCHAR(254)	NOT NULL	Email address	aparna@gmail.com
role	VARCHAR(10)	NOT NULL	User access role	ADMIN
department_id	BIGINT	FK, NULL	Department reference	1

is_active	BOOLEAN	NOT NULL	Account active status	1
is_staff	BOOLEAN	NOT NULL	Django admin access	1
is_superuser	BOOLEAN	NOT NULL	Administrator privilege	1
date_joined	DATETIME	NOT NULL	Account creation time	2026-09-29 14:31
last_login	DATETIME	NULL	Last login time	2026-10-24 16:02

3.9.2 Asset Table

Table 3.7 stores physical inventory assets, category classifications, and quantity thresholds.

Table 3.7: Asset table schema and sample data

Field Name	Data Type	Key/Constraint	Description	Sample Data
id	INTEGER	PK, Auto Increment	Unique asset ID	15
name	VARCHAR(200)	NOT NULL	Asset name/model	ThinkPad T14
category_id	BIGINT	FK, NOT NULL	Linked category	1
department_id	BIGINT	FK, NULL	Linked department	1
description	TEXT	BLANK	Specifications	Core i7, 16GB RAM
total_quantity	INTEGER	NOT NULL	Total fleet units	10
available_quantity	INTEGER	NOT NULL	Available in stock	9

low_stock_threshold	INTEGER	NOT NULL	Alert trigger limit	2

3.9.3 Asset Request Table

Table 3.8 stores asset checkout requisitions, approval status, and return timestamps.

Table 3.8: Asset request table schema and sample data

Field Name	Data Type	Key/Constraint	Description	Sample Data
id	INTEGER	PK, Auto Increment	Unique request ID	3
employee_id	BIGINT	FK, NOT NULL	Requesting user	23
asset_id	BIGINT	FK, NOT NULL	Requested asset	15
quantity	INTEGER	NOT NULL	Requested count	1
status	VARCHAR(10)	NOT NULL	Requisition state	APPROVED
notes	TEXT	BLANK	Checkout reason	Onsite deployment
admin_notes	TEXT	BLANK	Admin feedback	Approved for 14d
request_date	DATETIME	NOT NULL	Submission date	2026-09-30 03:58
response_date	DATETIME	NULL	Decision date	2026-09-30 04:04
return_date	DATETIME	NULL	Return timestamp	NULL

3.9.4 Department Table

Table 3.9 stores organizational departments.

Table 3.9: Department table schema and sample data

Field Name	Data Type	Key/Constraint	Description	Sample Data
id	INTEGER	PK, Auto Increment	Unique department ID	1
name	VARCHAR(100)	UNIQUE, NOT NULL	Department name	Finance
description	TEXT	BLANK	Department overview	Accounting & Payroll

3.9.5 Task and SubTask Table

Table 3.10 stores tasks and checklist subtasks.

Table 3.10: Task and SubTask table schema and sample data

Field Name	Data Type	Key/Constraint	Description	Sample Data
id	INTEGER	PK, Auto Increment	Unique task ID	1
title	VARCHAR(200)	NOT NULL	Task title	Deploy Server
assigned_to_id	BIGINT	FK, NULL	Assigned employee	19
workload_hours	INTEGER	NOT NULL	Estimated effort	3
priority	VARCHAR(10)	NOT NULL	Priority rank	HIGH
status	VARCHAR(20)	NOT NULL	Completion status	IN_PROGRESS
deadline	DATETIME	NOT NULL	Due timestamp	2026-10-02 18:03
created_at	DATETIME	NOT NULL	Creation timestamp	2026-10-02 18:03

3.9.6 Maintenance Issue Table

Table 3.11 stores equipment damage reports and repair records.

Table 3.11: Maintenance issue table schema and sample data

Field Name	Data Type	Key/Constraint	Description	Sample Data
id	INTEGER	PK, Auto Increment	Unique issue ID	1
asset_id	BIGINT	FK, NOT NULL	Associated asset	15
department_id	BIGINT	FK, NULL	Target department	1
reported_by_id	BIGINT	FK, NOT NULL	Reporter user ID	19
description	TEXT	NOT NULL	Defect details	Trackpad sticking
is_damaged	BOOLEAN	NOT NULL	Severe damage flag	1
priority	VARCHAR(10)	NOT NULL	Urgency level	HIGH
status	VARCHAR(20)	NOT NULL	Resolution status	REPORTED
reported_at	DATETIME	NOT NULL	Report timestamp	2026-10-04 06:23

3.9.7 Department Issue Table

Table 3.12 tracks inter-departmental collaboration requests and complaints.

Table 3.12: Department issue table schema and sample data

Field Name	Data Type	Key/Constraint	Description	Sample Data
id	INTEGER	PK, Auto Increment	Unique ticket ID	1
title	VARCHAR(200)	NOT NULL	Ticket title	Onboarding kit
description	TEXT	NOT NULL	Request details	Need developer badge
target_department_id	BIGINT	FK, NOT NULL	Target unit ID	2
reported_by_id	BIGINT	FK, NOT NULL	Reporter ID	20
priority	VARCHAR(10)	NOT NULL	Urgency ranking	MEDIUM
status	VARCHAR(20)	NOT NULL	Ticket state	OPEN
created_at	DATETIME	NOT NULL	Creation time	2026-09-29 15:26

3.9.8 Award Table

Table 3.13 stores achievement badges and recognitions.

Table 3.13: Award and badge table schema and sample data

Field Name	Data Type	Key/Constraint	Description	Sample Data
id	INTEGER	PK, Auto Increment	Unique badge ID	1
employee_id	BIGINT	FK, NOT NULL	Recipient ID	19
badge_icon	VARCHAR(50)	NOT NULL	Symbolic emoji	■
name	VARCHAR(100)	NOT NULL	Badge title	Timely Finish
description	TEXT	BLANK	Achievement note	Completed on time
awarded_at	DATETIME	NOT NULL	Timestamp	2026-10-05 13:13

Chapter 4. Results and Discussions

4.1 Results

The Role-Based Inventory and Asset Management System was developed and tested module by module. Each role has its own dashboard with a navigation sidebar and summary cards. Figures 4.1–4.6 show the implemented interfaces.

Figure 4.1 Users enter their username and password. Employees who do not have an account can register through the "Register" link.

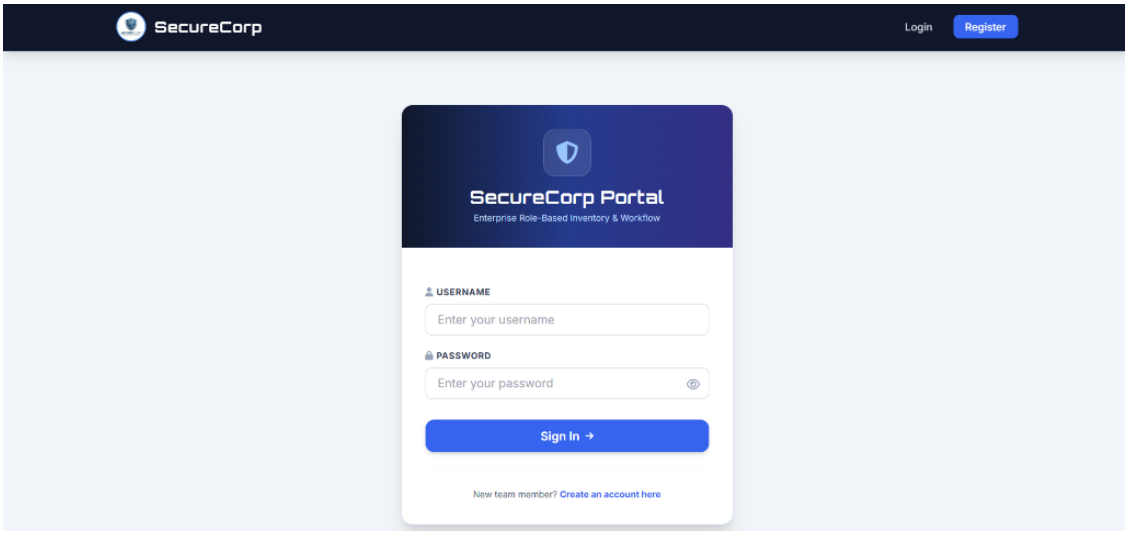


Figure 4.1: Login Page

Figure 4.2 Summary cards show the number of assets, low stock alerts, pending requests, active tasks, open issues and departments. A low stock alert section shows items below threshold. Quick action buttons allow adding assets, departments, creating tasks, granting awards, and generating PDF/Excel reports.

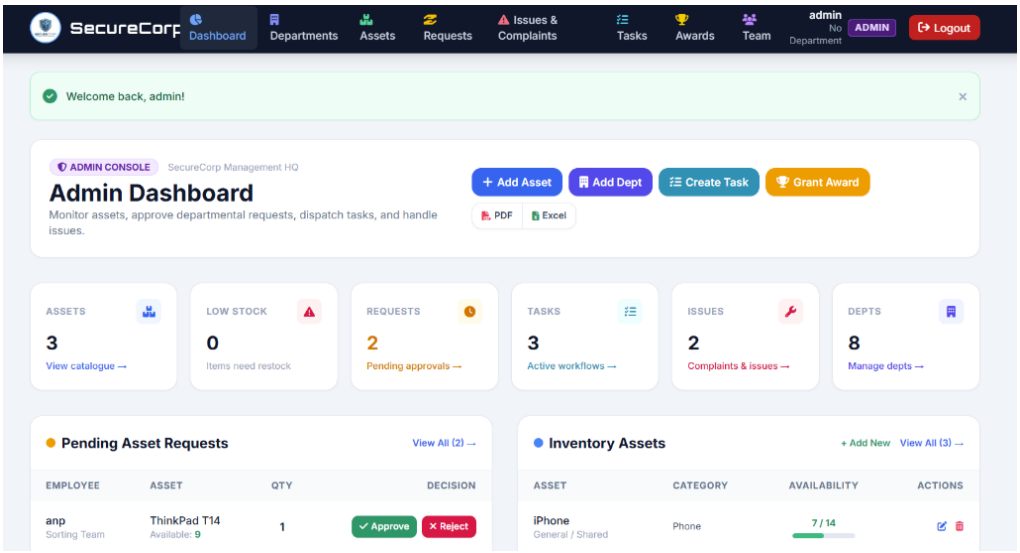


Figure 4.2: Admin Dashboard

Figure 4.3 Shows departmental units with staff headcount, allocated assets and issue counts. The administrator can create new departments and manage organizational structures.

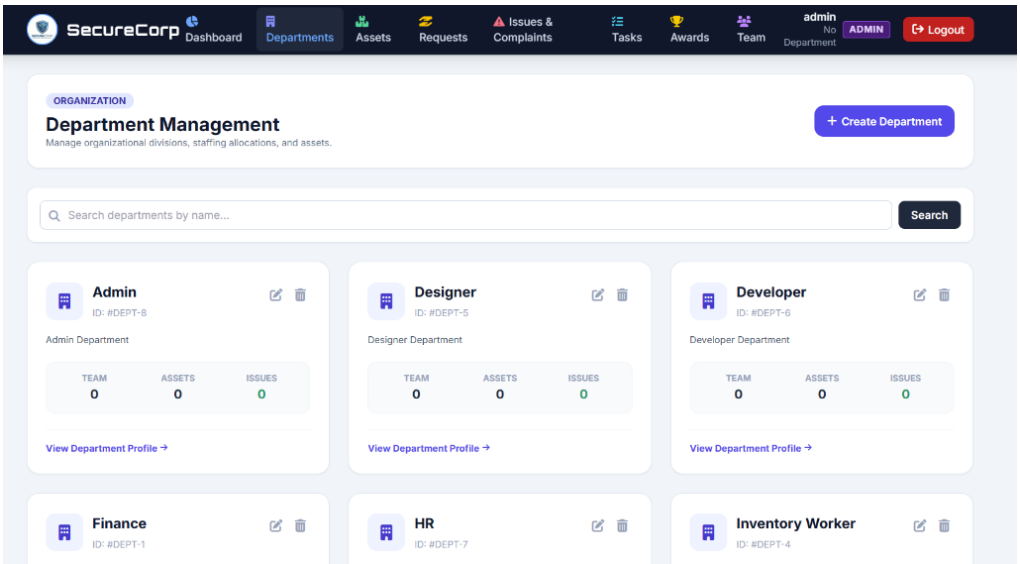


Figure 4.3: Department Management

Figure 4.4 Lists inventory assets with category, department, availability bar and stock status pills. Filter controls allow filtering by category, department and availability status.

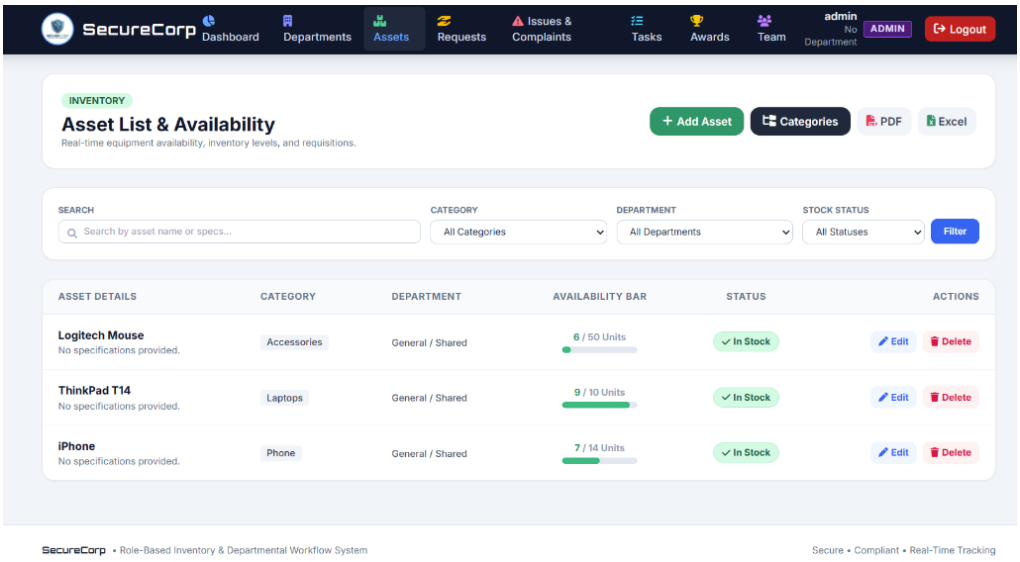


Figure 4.4: Asset List & Availability

Figure 4.5 Shows the asset creation form where administrators specify asset name, category, assigned department, total quantity, available quantity, and low-stock alert threshold.

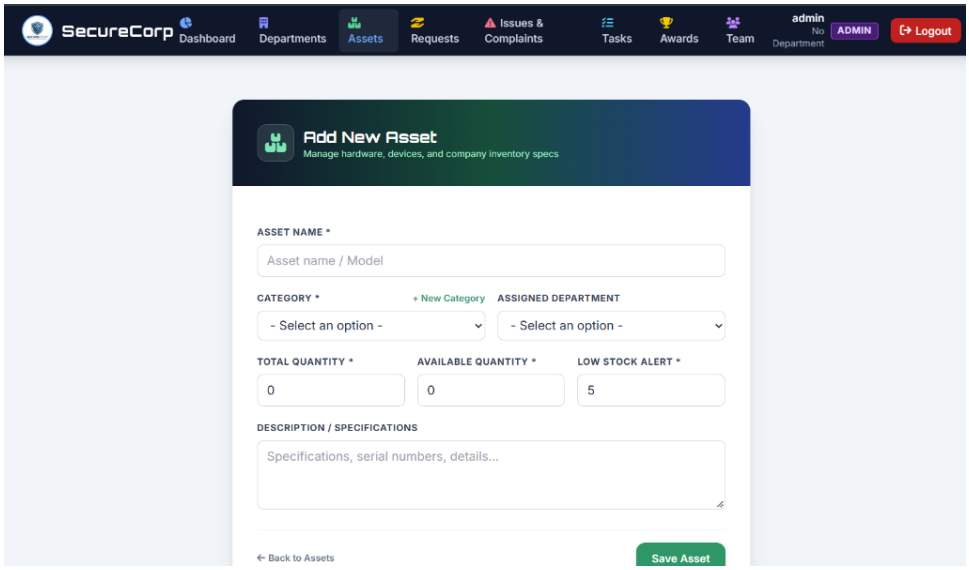


Figure 4.5: Add New Asset Form

Figure 4.6 Shows the employee dashboard where staff view active tasks, requisition approved assets, check off milestone subtasks, report maintenance defects, and track earned recognition badges.

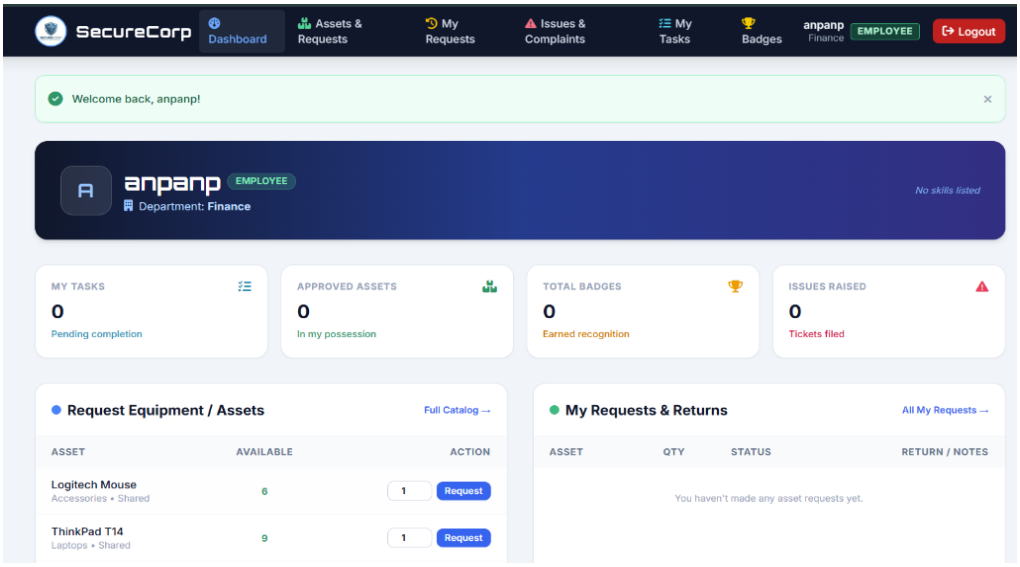


Figure 4.6: Employee Dashboard

4.2 Discussion

The results show that the Role-Based Inventory and Asset Management System manages the complete asset lifecycle through a single platform, from requisition submission to return restoration. Each user works through a dashboard that matches their role, and the status of requests, reports and inventory levels is available without approaching several people. This section discusses the observed behaviour of the system, its effect on the inventory monitoring process and its limitations.

4.2.1 Analysis of Results

The system replaces the email- and paper-based process with a single workflow. An employee submits a requisition, the administrator reviews it, and stock is decremented upon approval. When the item is returned, stock is automatically restored. The dashboards show the current stage of every request, so employees no longer need to approach several people to learn their status.

4.2.2 Role-Based Access Control

After login each user is redirected to the dashboard of their own role. If a user tries to open a page belonging to another role, they are redirected back to their own dashboard. The administrator is directed to the executive dashboard. This keeps data and functions limited to the people responsible for them.

4.2.3 Requisition Workflow and Validation

An employee cannot request more units than currently available in stock. Form validation prevents invalid quantities, and the return check-in action safely replenishes inventory back to shelf.

4.2.4 Development Process

The system was developed in an Agile manner, with separate feature branches for authentication, inventory, departments, requisitions and task tracking. Each branch was merged into the main branch after it was completed, so every module could be built and tested on its own without affecting the rest of the system.

4.2.5 Limitations

- Email and SMS notifications are not yet available, so users see updates on request status and task assignments only after logging in.
- Communication between the employee and the administrator is limited to stored ticket notes, with no real-time chat.
- The system currently runs in a development environment with sample data and has not yet been deployed on cloud servers for production.
- Automated barcode or RFID scanning hardware is not yet integrated for physical checkout counters.
- System-wide testing with a large number of concurrent users and legacy data imports is still in progress.

Chapter 5. Conclusion and Future Scope

5.1 Conclusion

The Role-Based Inventory and Asset Management System is a web-based application that brings the management, monitoring and requisition of departmental equipment into one centralized platform. It replaces the email- and paper-based process with a role-based system for Administrators, Department Heads, and Employees. Each user logs in to a dashboard that shows only the features and information relevant to their role.

The system covers the complete asset lifecycle, from inventory cataloging and threshold warnings to requisition approval, return stock restoration, defect reporting, and workload-balanced task dispatching. Because asset status, rejection notes and return logs are stored in one place, employees no longer need to approach several people to know the status of their requests, and the problems of lost, delayed or duplicated information are greatly reduced. Overall, the system improves communication between staff and administrators, reduces paperwork and manual effort, and makes the inventory tracking process more transparent and reliable. In future, it can be improved by adding email and SMS notifications, real-time messaging, barcode scanning, and deployment for use by a whole institution.

Key achievements include:

- Secure role-based authentication with a dashboard for each role.
- An online requisition workflow with stock verification, approval, rejection feedback, and automated return restoration.
- Department management and dual-channel maintenance and inter-departmental ticketing.
- A structured database holding asset records, requests, tasks, subtasks, issues and awards.

- Workload auto-assignment balancing tasks across eligible employees.
- An Agile development process with version-controlled, feature-wise development using Git and GitHub.

The project demonstrates how web technologies and a role-based workflow can be integrated to address the problem of scattered and manual inventory management in organizational departments. The system can assist employees in tracking their requisitions, defects and tasks, and help administrators to monitor and evaluate assets efficiently by keeping all inventory records in one centralized platform.

5.2 Future Scope

The proposed system can be further enhanced to improve its accuracy, usability, and practical application. The following features can be considered in future versions:

- **Barcode and QR code scanning:** Add mobile camera scanning for fast equipment checkout and check-in.
- **Email and SMS notifications:** Notify employees and administrators about status changes, stock alerts and task deadlines.
- **Real-time chat:** Add direct messaging between employees and administrators.
- **Deadline reminders and calendar:** Add automatic reminders for pending tasks and equipment return schedules.
- **Analytics and reports:** Provide charts and exportable reports of asset depreciation and maintenance costs for the department.
- **Production deployment:** Move from SQLite to PostgreSQL and host the system on cloud servers for the whole organization.
- **Mobile application:** Provide a mobile version for quick inventory scanning and request updates.

References

[1] O. Eweoya, A. O. Agbeyangi and J. M. Lukose, “TaskHive: A Web-Based Academic Project Management System for Higher Education,” Information Impact: Journal of Information and Knowledge Management, 2025. [Online]. Available: https://www.ajol.info/index.php/iijikm/article/view/302689

[2] IIT Madras, “Tracking Student Progress in Software Projects,” Software Engineering Projects. [Online]. Available: https://seprojects.study.iitm.ac.in/term_projects/2024_sept/problem_statement.html

[3] GitHub Docs, “GitHub Documentation.” [Online]. Available: https://docs.github.com/

[4] Django Software Foundation, “Django Documentation.” [Online]. Available: https://docs.djangoproject.com/

[5] Tailwind Labs, “Tailwind CSS Documentation,” 2026. [Online]. Available: https://tailwindcss.com/

[6] Python Software Foundation, “Python 3.13 Documentation,” 2026. [Online]. Available: https://docs.python.org/3/

Appendix A: Data Flow Diagram

Figure A.1 Level 0 DFD shows the Role-Based Inventory and Asset Management System as a single process, with the Employee, Administrator and Department Head as external entities and the database as the data store.

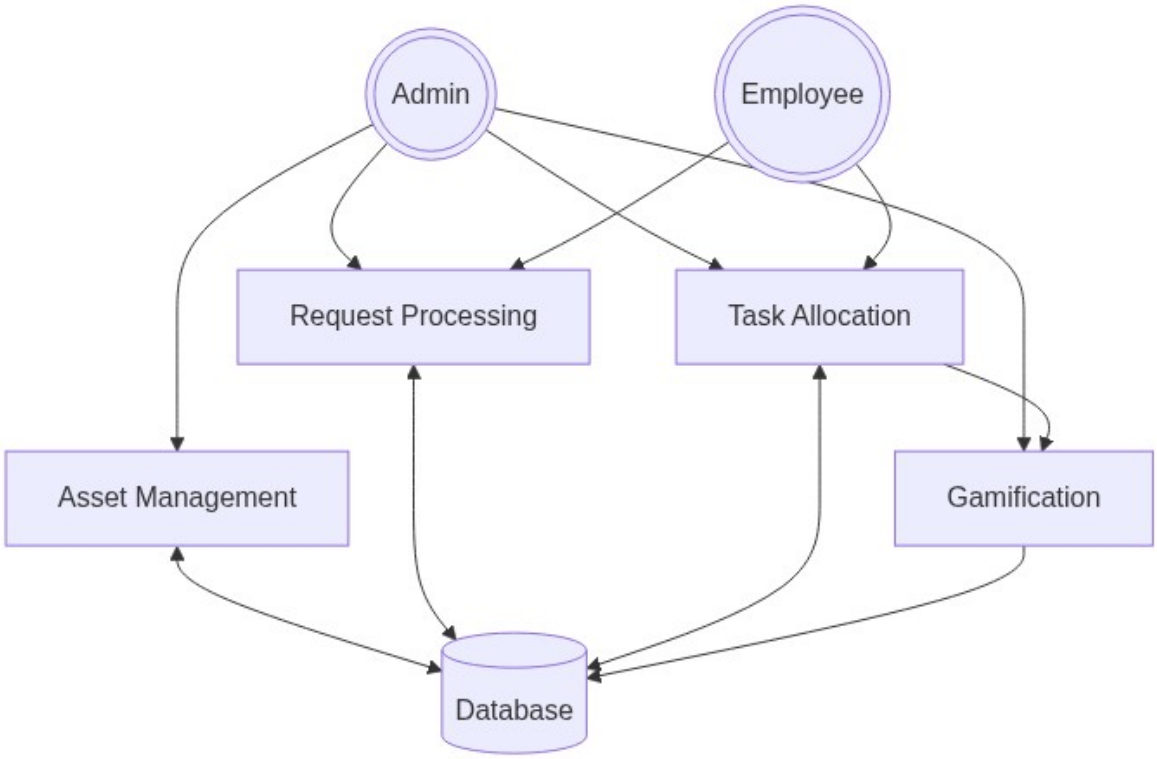


Figure A.1: Level 0 DFD

Figure A.2 Level 1 DFD shows the processes for the Employee, such as registration, login, catalog browsing, asset requisition submission, return check-in, maintenance defect reporting and viewing of status and badges.

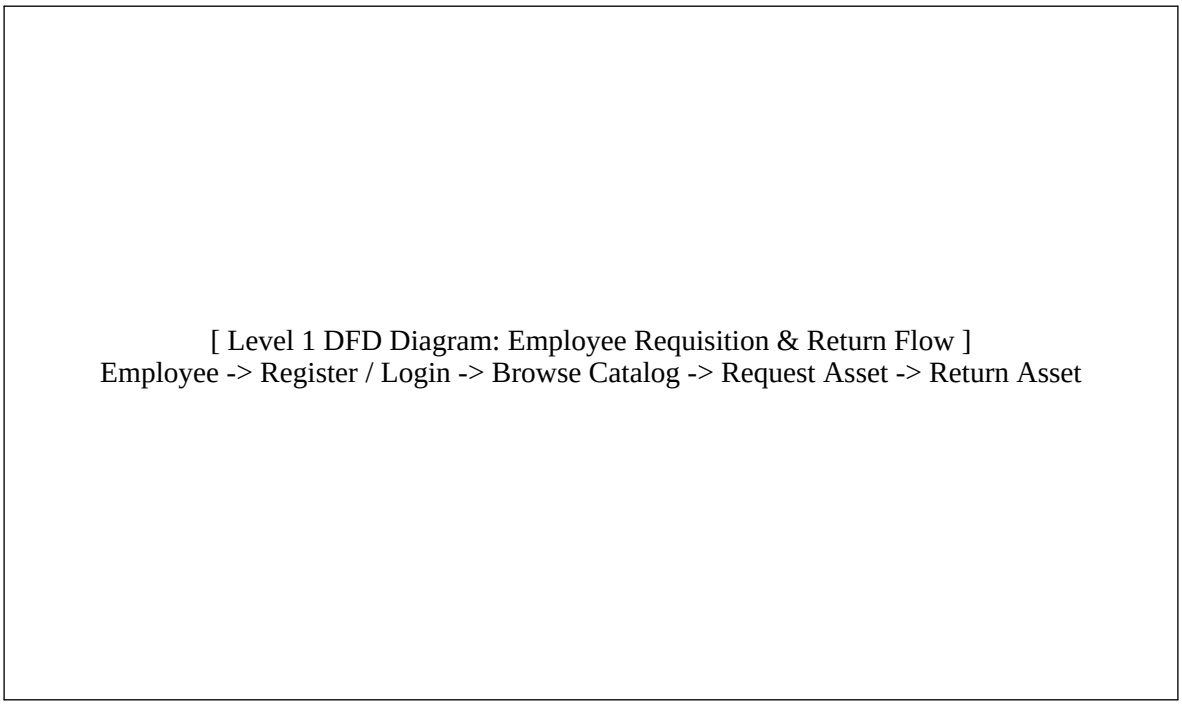


Figure A.2: Level 1 DFD

Figure A.3 Level 2 DFD shows the processes for the Administrator, such as managing assets, creating departments, approving or rejecting requisitions, allocating equipment, resolving maintenance issues, and dispatching tasks.

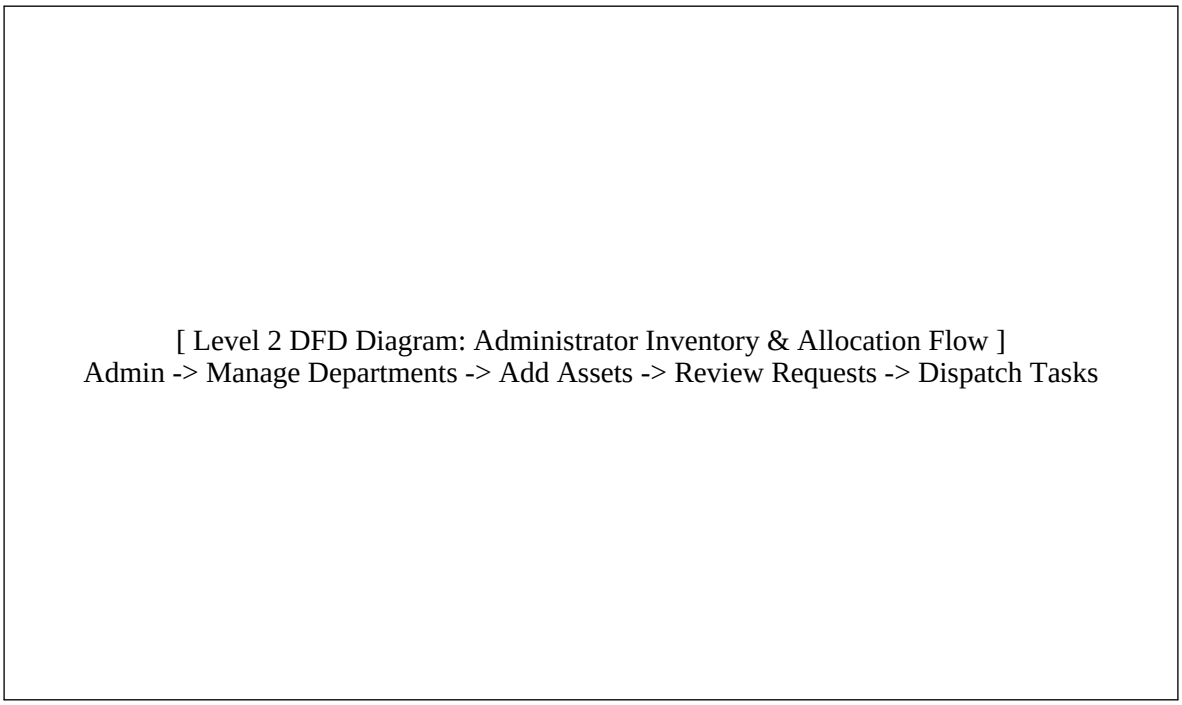


Figure A.3: Level 2 DFD

Figure A.4 Level 3 DFD shows the processes for Requisitions, checking inventory sufficiency, updating request status, decrementing available stock upon approval, and replenishing stock upon return.

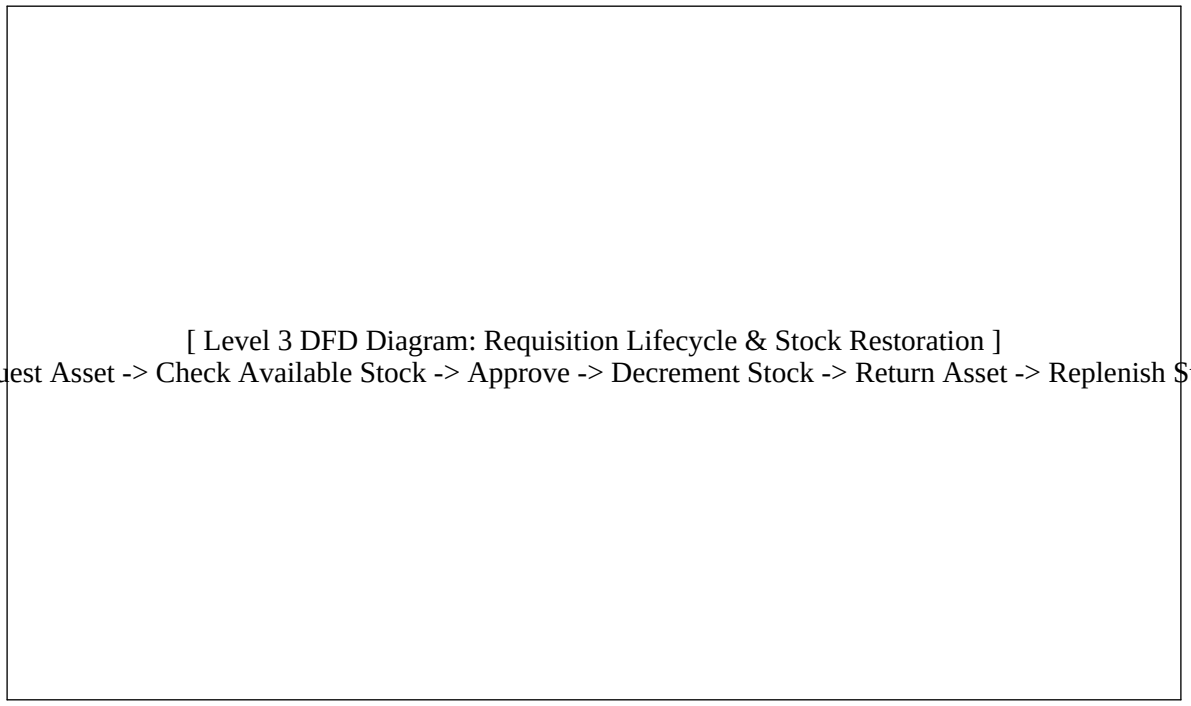


Figure A.4: Level 3 DFD

Figure A.5 Level 4 DFD shows the processes for Maintenance and Tasks, such as defect reporting, cross-department ticketing, workload-balancing auto-assignment, subtask toggling, and timely completion badge granting.

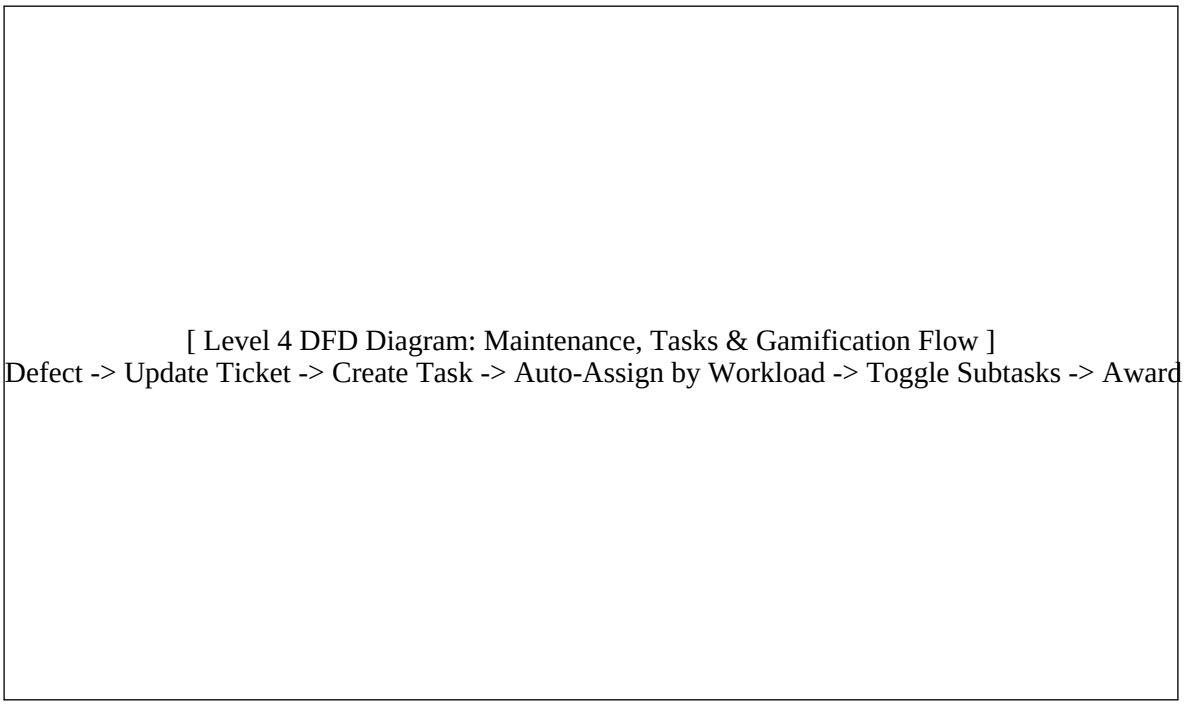


Figure A.5: Level 4 DFD

Appendix B: Entity–Relationship Diagram

Figure B.1 shows the entities, attributes and relationships used by the Role-Based Inventory and Asset Management System. It covers users and their roles, departments, asset categories, physical equipment items, asset requests, tasks, milestone subtasks, maintenance issues, department tickets, and recognition awards. The relationships show how assets belong to categories and departments, how requests link employees to assets, and how tasks and issues are tracked across departments.

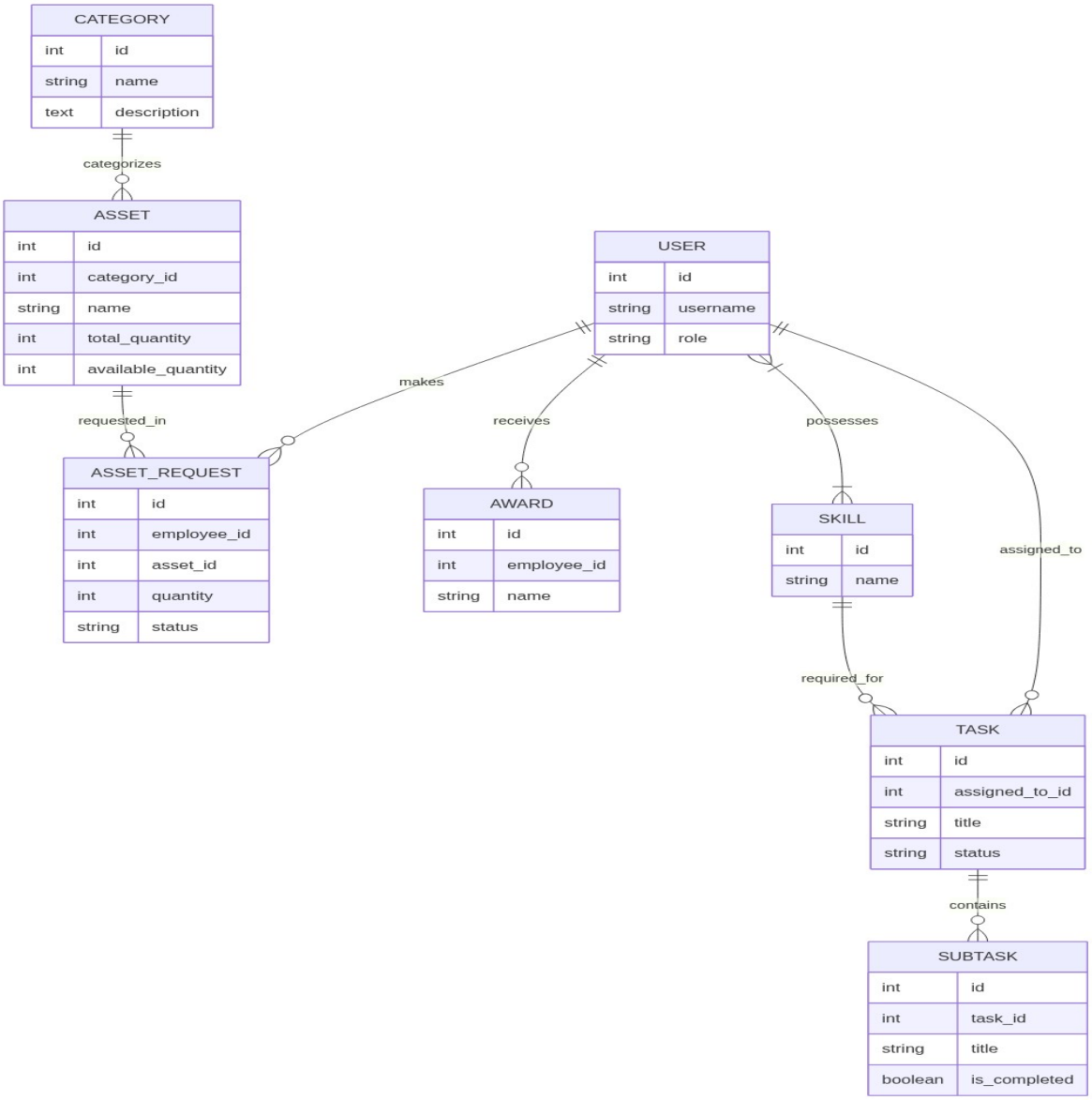


Figure B.1: Entity–relationship diagram for users, roles, assets, requests and tasks

Appendix C: Source Code

C.1 Asset Data Model

```
class Asset(models.Model):
    name = models.CharField(max_length=200)
    category = models.ForeignKey(Category, on_delete=models.CASCADE, related_name
='assets')
    department = models.ForeignKey(Department, on_delete=models.SET_NULL, null=Tr
ue, blank=True)
    description = models.TextField(blank=True)
    total_quantity = models.PositiveIntegerField(default=0)
    available_quantity = models.PositiveIntegerField(default=0)
    low_stock_threshold = models.PositiveIntegerField(default=5)
    def __str__(self):
        return self.name
    @property
    def is_low_stock(self):
        return self.available_quantity <= self.low_stock_threshold
    @property
    def stock_status(self):
        if self.available_quantity == 0:
            return 'OUT_OF_STOCK'
        elif self.available_quantity <= self.low_stock_threshold:
            return 'LOW_STOCK'
        return 'IN_STOCK'
```

C.2 Asset Requisition and Return Model

```
class AssetRequest(models.Model):
    STATUS_CHOICES = (
        ('PENDING', 'Pending'),
        ('APPROVED', 'Approved'),
        ('REJECTED', 'Rejected'),
        ('RETURNED', 'Returned'),
    )
    employee = models.ForeignKey(User, on_delete=models.CASCADE, related_name='re
quests')
    asset = models.ForeignKey(Asset, on_delete=models.CASCADE, related_name='requ
ests')
    quantity = models.PositiveIntegerField(default=1)
    notes = models.TextField(blank=True, default='')
    admin_notes = models.TextField(blank=True, default='')
    status = models.CharField(max_length=10, choices=STATUS_CHOICES, default='PEN
DING')
    request_date = models.DateTimeField(auto_now_add=True)
    response_date = models.DateTimeField(null=True, blank=True)
    return_date = models.DateTimeField(null=True, blank=True)
    def __str__(self):
        return f'{self.employee.username} - {self.asset.name} ({self.status})'
```

C.3 Task and SubTask Models

```
class Task(models.Model):
    STATUS_CHOICES = (
        ('PENDING', 'Pending'),
        ('IN_PROGRESS', 'In Progress'),
        ('COMPLETED', 'Completed'),
    )
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    assigned_to = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, b
lank=True)
    required_skill = models.ForeignKey(Skill, on_delete=models.SET_NULL, null=Tru
e, blank=True)
    deadline = models.DateTimeField()
    workload_hours = models.PositiveIntegerField(default=1)
    priority = models.CharField(max_length=10, default='MEDIUM')
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='PEN
DING')
    created_at = models.DateTimeField(auto_now_add=True)
    completed_at = models.DateTimeField(null=True, blank=True)
class SubTask(models.Model):
    task = models.ForeignKey(Task, on_delete=models.CASCADE, related_name='subtas
ks')
    title = models.CharField(max_length=200)
    is_completed = models.BooleanField(default=False)
```

C.4 Admin Requisition Approval View

```
@login_required
def approve_request(request, request_id):
    if not request.user.is_admin_role:
        return redirect('employee_dashboard')
    asset_req = get_object_or_404(AssetRequest, id=request_id)
    if asset_req.status == 'PENDING':
        if asset_req.asset.available_quantity >= asset_req.quantity:
            asset_req.asset.available_quantity -= asset_req.quantity
            asset_req.asset.save()
            asset_req.status = 'APPROVED'
            asset_req.response_date = timezone.now()
            asset_req.save()
            messages.success(request, f'Request #{asset_req.id} approved successf
ully.')
        else:
            messages.error(request, 'Not enough quantity available.')
    return redirect('request_list')
```

C.5 Return Asset Controller View

```
@login_required
def return_asset(request, request_id):
    asset_req = get_object_or_404(AssetRequest, id=request_id)
    if not request.user.is_admin_role and request.user != asset_req.employee:
        return redirect('employee_dashboard')
    if asset_req.status == 'APPROVED':
        asset_req.asset.available_quantity += asset_req.quantity
        if asset_req.asset.available_quantity > asset_req.asset.total_quantity:
            asset_req.asset.available_quantity = asset_req.asset.total_quantity
        asset_req.asset.save()
        asset_req.status = 'RETURNED'
        asset_req.return_date = timezone.now()
        asset_req.save()
        messages.success(request, f'Asset returned. Stock restored.')
    return redirect('request_list')
```

C.6 Workload Auto-Assignment View

```
if 'auto_assign' in request.POST and task.required_skill:
    eligible = User.objects.filter(role='EMPLOYEE', skills=task.required_skill)
    if eligible.exists():
        eligible = eligible.annotate(
            workload=Sum('tasks__workload_hours', filter=Q(tasks__status__in=['PE
NDING', 'IN_PROGRESS']))
        )
        best_emp = None
        lowest_workload = float('inf')
        for emp in eligible:
            wl = emp.workload or 0
            if wl < lowest_workload:
                lowest_workload = wl
                best_emp = emp
        if best_emp:
            task.assigned_to = best_emp
            messages.success(request, f'Auto-assigned to {best_emp.username}.')
```

C.7 Subtask Milestone Toggle Controller

```
@login_required
def update_subtask_status(request, subtask_id):
    subtask = get_object_or_404(SubTask, id=subtask_id)
    task = subtask.task
    if request.method == 'POST':
        subtask.is_completed = request.POST.get('is_completed') == 'on'
        subtask.save()
        all_done = all(st.is_completed for st in task.subtasks.all())
        if all_done and task.status != 'COMPLETED':
            task.status = 'COMPLETED'
            task.completed_at = timezone.now()
            task.save()
            if task.completed_at <= task.deadline and task.assigned_to:
                Award.objects.create(
                    employee=task.assigned_to,
                    badge_icon='■',
                    name=f'Timely Finish: {task.title[:30]}',
                    description=f'Completed on time before deadline.'
                )
    return redirect('task_list')
```

C.8 System URL Configurations (inventory/urls.py)

```
urlpatterns = [
    path('', views.dashboard_view, name='dashboard'),
    path('login/', views.custom_login_view, name='login'),
    path('register/', views.register, name='register'),
    path('logout/', views.custom_logout_view, name='logout'),
    path('admin-dashboard/', views.admin_dashboard, name='admin_dashboard'),
    path('employee-dashboard/', views.employee_dashboard, name='employee_dashboar
d'),
    path('departments/', views.department_list, name='department_list'),
    path('assets/', views.asset_list, name='asset_list'),
    path('requests/', views.request_list, name='request_list'),
    path('issues/', views.issue_list, name='issue_list'),
    path('tasks/', views.task_list, name='task_list'),
    path('awards/', views.award_list, name='award_list'),
]
```

Appendix D: Screenshots

D.1 Login Page

Figure D.1 shows the login interface where administrators, department heads, and employees authenticate securely with role-based automatic routing.

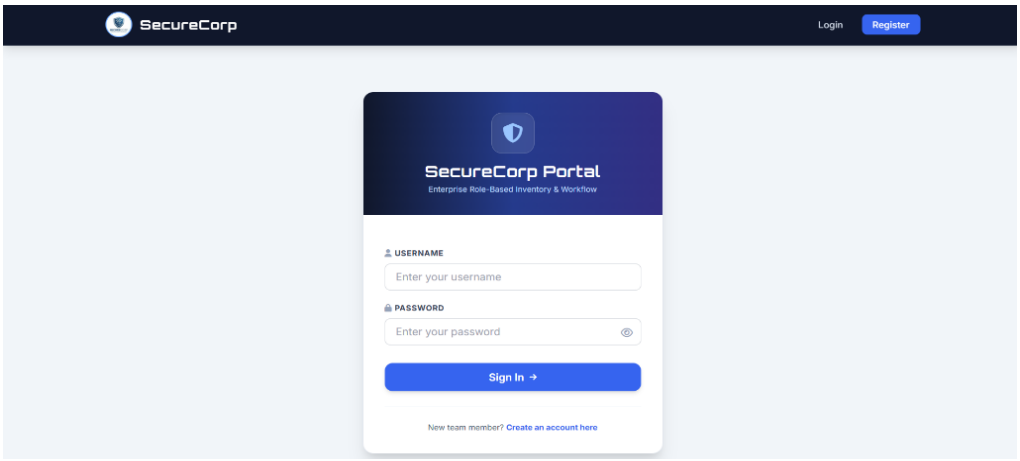


Figure D.1: Login page of the system

D.2 Admin Dashboard Page

Figure D.2 shows the administrator dashboard with real-time inventory metrics, low-stock threshold warning counters, active requisition queues, and administrative action buttons.

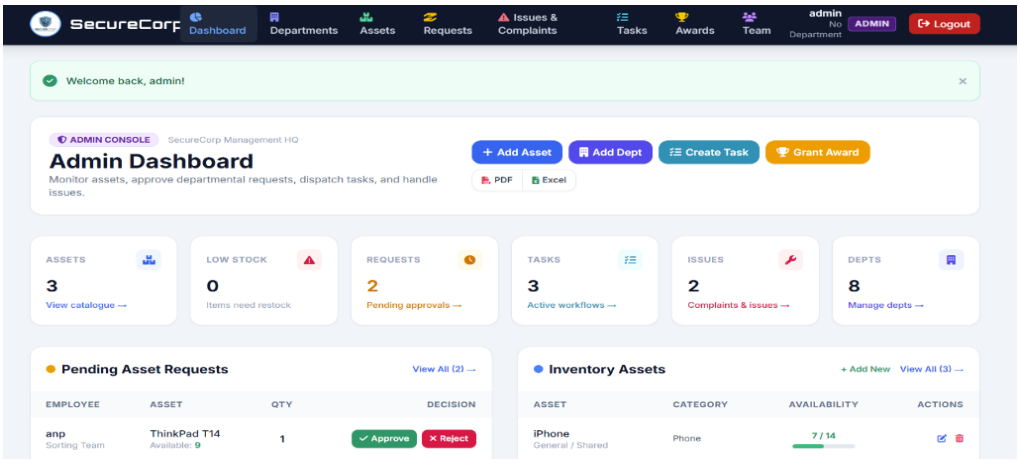


Figure D.2: Admin dashboard with asset metrics

D.3 Department Management Page

Figure D.3 shows the department management directory displaying staff headcount, allocated departmental assets, issue counts, and controls to add or edit organizational units.

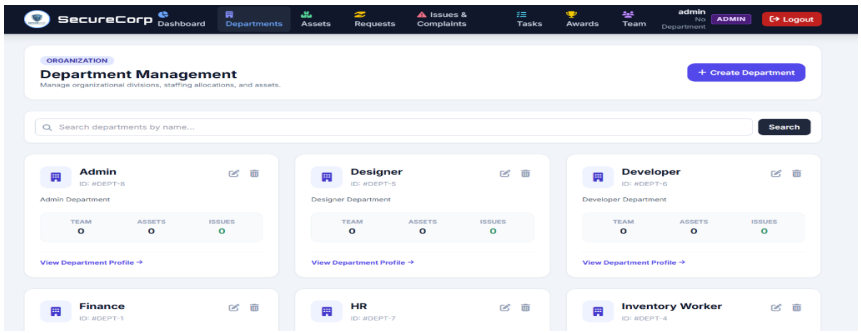


Figure D.3: Department management directory

D.4 Asset List & Availability Page

Figure D.4 shows the asset catalog with real-time availability progress bars, category classifications, department assignments, and quick search filters.

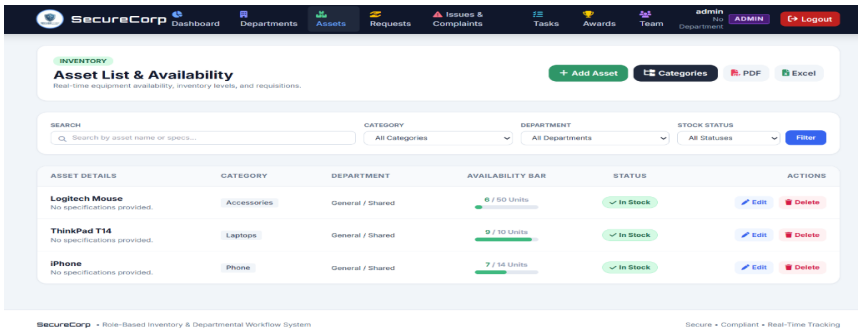


Figure D.4: Asset catalog with availability progress bars

D.5 Add New Asset Page

Figure D.5 shows the add new asset form where administrators specify item details, category, assigned department, initial inventory count, and low-stock alert thresholds.

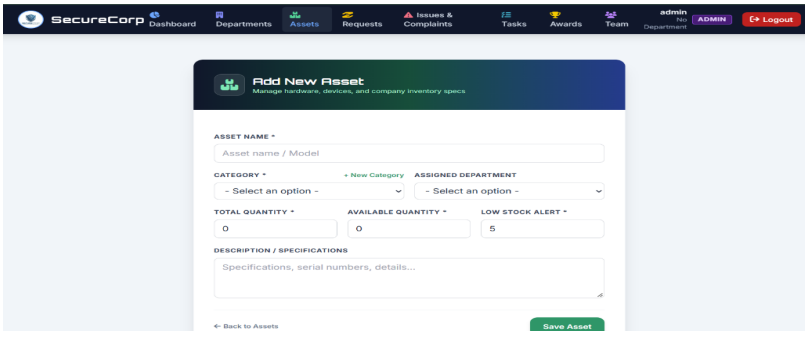


Figure D.5: Add new asset form

D.6 Employee Dashboard Page

Figure D.6 shows the employee workspace dashboard displaying assigned tasks, milestone subtask checkboxes, recent requisition history, and earned badges.

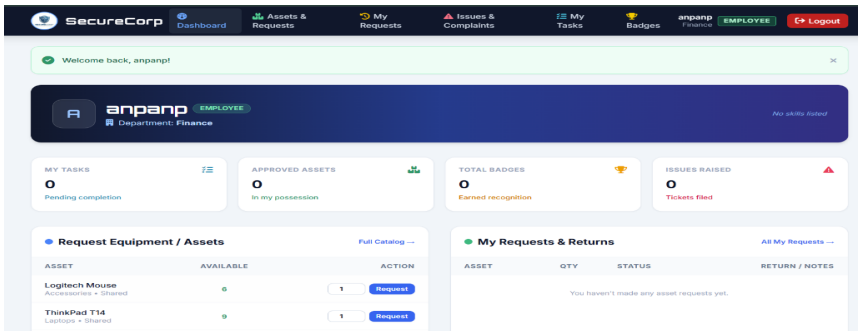


Figure D.6: Employee workspace dashboard

D.7 Asset Requests & Approvals Page

Figure D.7 shows the asset requisition and return tracker where administrators review requests, grant approvals or rejections, and check in returned items to replenish stock.

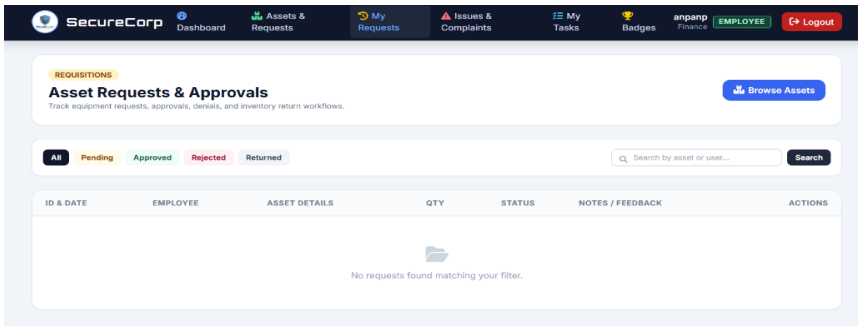


Figure D.7: Asset requisition and return tracker

D.8 Complaint & Issue Management Page

Figure D.8 shows the dual-channel issue tracking interface covering physical equipment maintenance defects and cross-departmental operational tickets through to resolution.

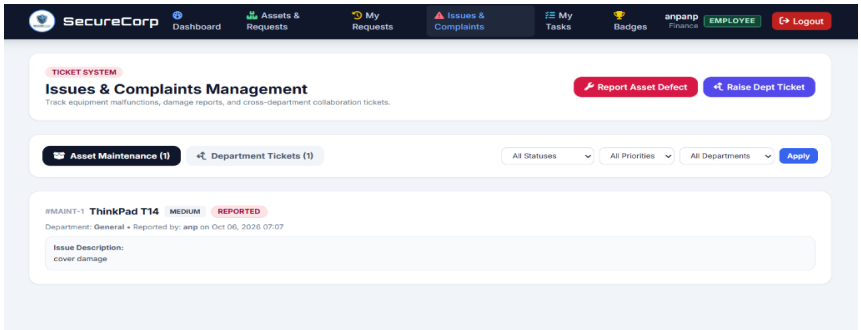


Figure D.8: Maintenance defect and issue tracker

D.9 Task & Subtask Management Page

Figure D.9 shows the workload-balanced task management board where administrative tasks are broken down into milestone subtasks, assigned to employees, and monitored.

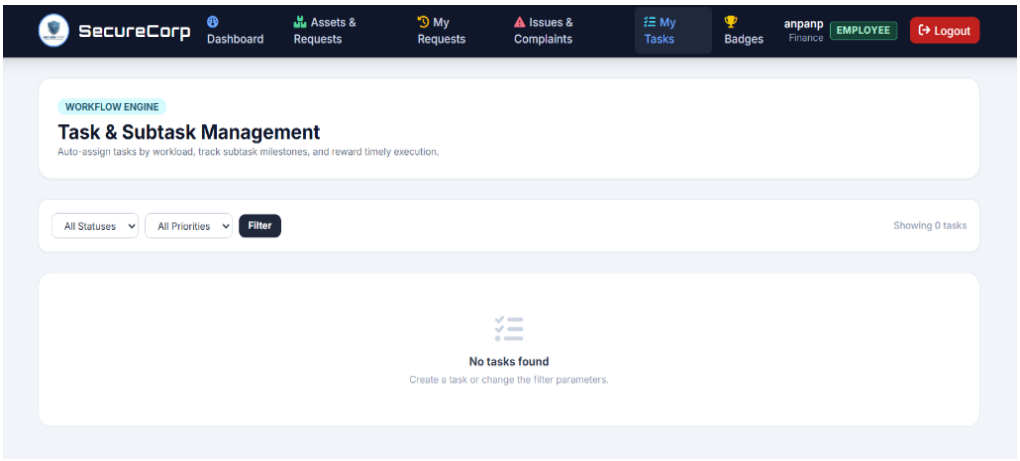


Figure D.9: Task and milestone subtask board

D.10 Performance Awards & Badges Page

Figure D.10 shows the gamified recognition center where employees earn achievement badges for exemplary contributions and timely milestone completions.

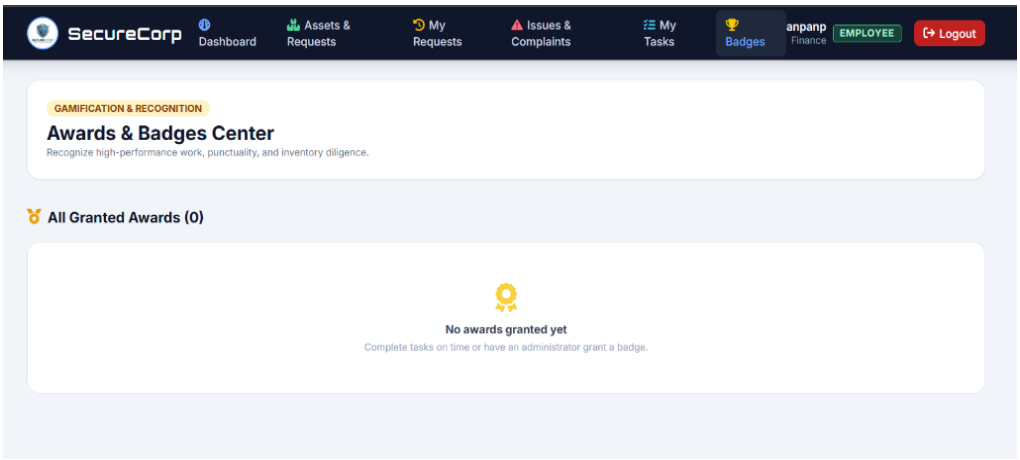


Figure D.10: Gamified recognition badges and awards